MALAYSIAN INSTITUTE OF INFORMATION TECHNOLOGY

Universiti Kuala Lumpur

ISB 46703 — Principles of Artificial Intelligence

Comprehensive Mini-Project: Multi-Agent System Design (25%)

PROJECT SENTINEL-4

A Multi-Agent Digital Countermeasure Unit Against the Rogue AI Threat "The Entity"

Theme / Domain: Cyber Defence & Synthetic-Media Threat Analysis

Orchestration framework: **LangGraph** · Pattern: **Hierarchical supervisor over a shared-state blackboard**
5 specialised agents · 5 custom tools · 4 conditional routing paths

No.	Group Member	Student ID
1	**HARRIS ILHAM SHAH BIN MOHD AZRAF** Group Leader	52213125949
2	DHARIZMAN BIN GHAZALI @ HASSAN	52213125618
3	AZAD ARIF BIN ABDUL RAZAK	52213125884
4	MUHAMMAD NAZRUL AIMAN BIN MOHD NIZAM	52213124200

Lecturer:	Dr. Chen
Submission Date:	10 September 2026 (Week 7)
Repository:	github.com/DHARIZMAN/sentinel4-mas

Table of Contents

> **How to read this report.** Every architectural claim in this document is anchored to a specific file and line in the submitted source code, written as `file.py:line`. Every performance figure in Section 5 and 6 was produced by executing the submitted code; the commands that reproduce each figure are given alongside it.

1. Project Overview

1.1 Problem framing

An adversary that generates synthetic media and executes network intrusion *simultaneously* defeats the traditional Security Operations Centre model, in which a human analyst serially consults one specialist tool after another. By the time a voice-cloning verdict returns, the credential obtained by that cloned voice has already been used. The defensive response has to be as parallel and as automated as the attack.

Project **SENTINEL-4** models that response. It is a Multi-Agent System that receives a free-text incident brief from a duty operator, dynamically decides which forensic specialists the incident actually requires, runs the independent ones concurrently, fuses their findings into a single verified attack vector, and — only when the fused severity warrants it — commissions a predictive counter-strategy. The adversary is codenamed **"The Entity"**, the rogue autonomous system named in the assessment brief.

1.2 System at a glance

Property	Implementation
Orchestration framework	LangGraph `StateGraph` (`src/graph.py:32` , compiled at `src/graph.py:302`)
Theme / domain	Digital countermeasure unit vs. rogue AI "The Entity" (recommended theme)
Collaboration pattern	Hierarchical supervisor over a shared-state blackboard
Router type	**DYNAMIC** — LLM-driven semantic intent classification, with a deterministic static keyword fallback
Specialised agents	5 (assessment minimum: 3)
Custom tools	5 (assessment minimum: 3)
Conditional routing paths	4 (assessment minimum: 2)
LLM request timeout	25 s default, hard-clamped to ≤ 30 s (`src/config.py:240`)
LLM max retries	2 default, hard-clamped to ≤ 3 (`src/config.py:241`)
Exception handlers	18 handlers across 14 distinct exception types (minimum: 3)
Docstring coverage	100% — 173 of 173 modules, classes, methods and functions
Human-review annotations	13 `[HUMAN-REVIEW]` comments (minimum: 5)
Automated tests	35, all passing in 4.79 s
Offline operation	Deterministic mock engine — the whole system runs and is gradeable with no API key and no network

1.3 Handling complex, multi-step input

The system accepts an unstructured operator brief plus an optional list of evidence artefacts. The shipped scenario `scenarios/scenario_multi_vector.json` is an 810-character brief describing three simultaneous attack vectors (a synthetic CFO voicemail, manipulated server-room CCTV, and outbound C2 beaconing exploiting a named CVE) accompanied by two media artefacts. Processing this single input requires the

system to take fourteen discrete reasoning steps across six agents, including one self-triggered refinement loop, before it releases a product.

1.4 Repository structure

```
entity_mas/
├── main.py                  CLI entry point (scenario / ad-hoc brief / JSON output)
├── demo_fallback.py         Live failure-injection demonstration (4 failure classes)
├── audit_requirements.py    Self-audit of the code against the assessment rubric
├── requirements.txt         Pinned runtime dependencies
├── Dockerfile               Reproducible container runtime
├── .env.example             Configuration template (no secrets)
├── .gitignore               Excludes .env, venv/, __pycache__/
├── src/
│   ├── config.py            Settings + embedding-model misrouting guard
│   ├── state.py             MissionState blackboard + concurrency reducers
│   ├── llm_client.py        Timeouts, retries, JSON repair, offline mock engine
│   ├── router.py            DYNAMIC semantic router + static fallback
│   ├── graph.py             StateGraph assembly, 4 conditional routing paths
│   ├── evaluation.py        Threat fusion + adversarial self-evaluation
│   ├── fallback.py          Risk warning + partial-response generator
│   ├── agents/              Abstract base + 4 specialists + standard defense
│   └── tools/               Registry + 5 custom tools
├── scenarios/               3 runnable demonstration scenarios
├── evidence/                Sandboxed artefacts for read_evidence_file
├── docs/                    Architecture diagram + this report
└── tests/                   35 tests: routing, tools, fallback latency
```

2. System Architecture & State Management

2.1 Architecture diagram

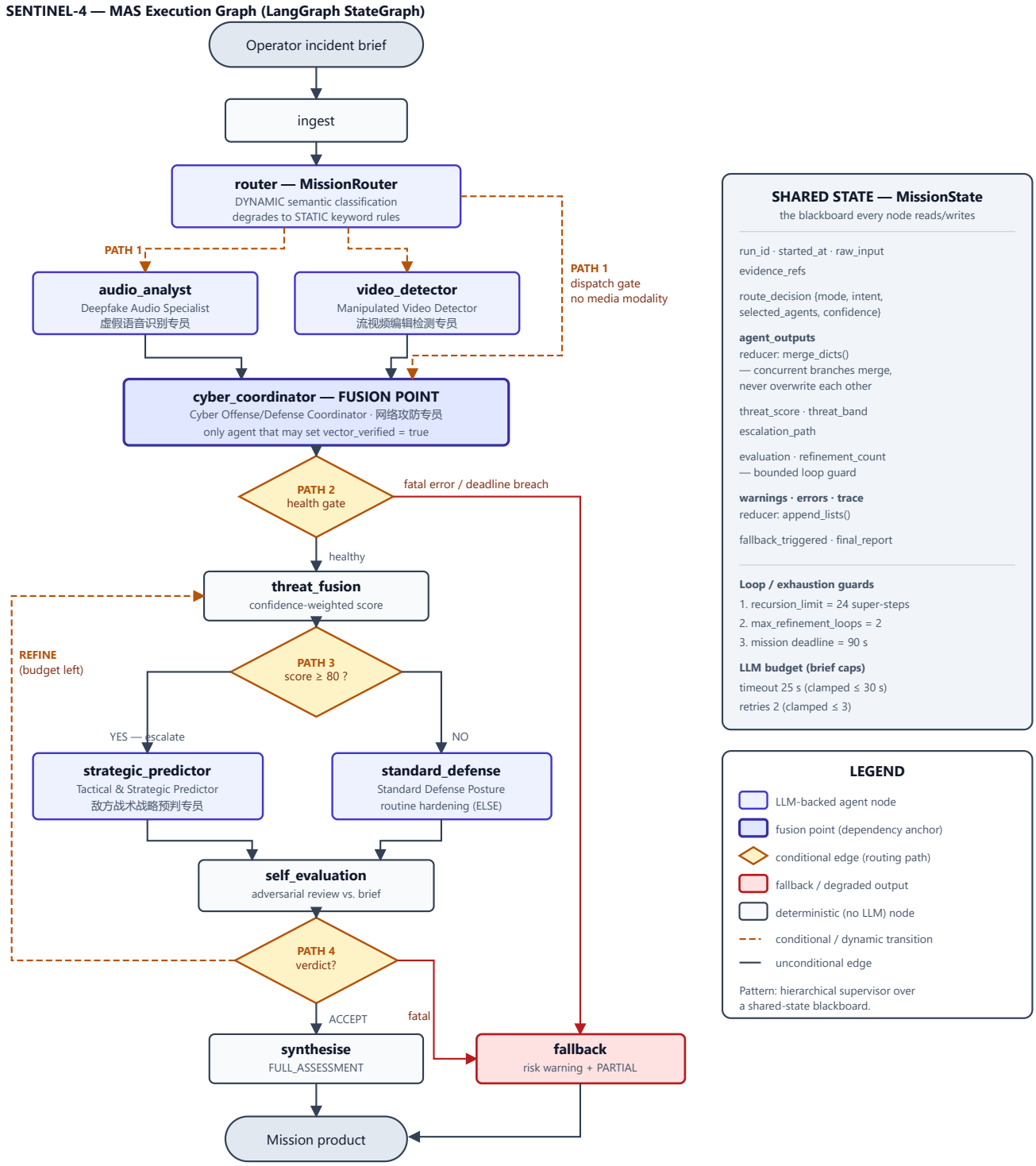

Figure 1 — SENTINEL-4 execution graph. Agent nodes (indigo), deterministic nodes (grey), conditional gates (amber diamonds) and the degraded-output path (red). The right-hand panel documents the shared `MissionState` blackboard, its concurrency reducers, and the three independent loop guards. Source: `docs/architecture.svg`; equivalent Mermaid source in `docs/architecture.mermaid`.

2.2 Node inventory

Node	Type	Responsibility	Source
`ingest`	Deterministic	Stamp run ID, record brief size and artefact count	`graph.py:98`
`router`	LLM	Semantic intent classification → specialist selection	`router.py:145`
`audio_analyst`	LLM + tools	Speech-audio authenticity verdict	`agents/audio_analyst.py:23`
`video_detector`	LLM + tools	Video manipulation verdict	`agents/video_detector.py:140`
`cyber_coordinator`	LLM + tools	Evidence fusion, attack-vector verification, containment	`agents/cyber_coordinator.py:18`
`threat_fusion`	Deterministic	Confidence-weighted fusion of specialist scores	`evaluation.py:116`
`escalation_gate`	Deterministic	Record which branch the threat gate selected	`graph.py:249`
`strategic_predictor`	LLM + tools	Adversary next-moves + pre-positioned counter-strategy	`agents/strategic_predictor.py:139`
`standard_defense`	LLM	Proportionate routine hardening (the ELSE branch)	`agents/standard_defense.py:243`
`self_evaluation`	LLM	Adversarial review of the draft against the original brief	`evaluation.py:177`
`synthesise`	Deterministic	Assemble the full assessment product	`graph.py:149`
`fallback`	Deterministic	Risk warning + partial response from salvaged data	`fallback.py:428`

2.3 State management — the blackboard

All twelve nodes read from and write to one `MissionState` TypedDict (`src/state.py:59`). Choosing an explicit shared state rather than point-to-point message passing is the decision that makes two required behaviours possible at all. First, a late agent (the Strategic Predictor) can consult the findings of *every* earlier agent without those agents knowing it exists. Second, the fallback handler can assemble a partial answer from whatever happens to be on the board at the moment of failure — there is no message queue to reconstruct.

2.3.1 Concurrency reducers

LangGraph applies a *reducer* when two branches write the same state key within one super-step. Because the Audio and Video specialists are dispatched concurrently and both write to `agent_outputs`, the default last-write-wins behaviour silently destroyed one of the two reports. The system therefore declares explicit reducers:

```
agent_outputs : Annotated[dict, merge_dicts]   # state.py:94  — union, right wins on key collision
activated_agents, warnings, errors, trace :
                Annotated[list, append_lists]  # state.py:92,103,104,108 — concatenate, never replace
```

> **Design intervention.** `merge_dicts` (`state.py:18`) is the single most important correctness fix in the state layer. Without it, whichever media specialist finished second erased the other's report, and the fused threat score was computed from half the evidence. The rationale is recorded in the source at `state.py:33–37`.

2.3.2 State channels

Channel	Purpose
`run_id`, `started_at`	Mission identity and the monotonic clock reading all elapsed-time and fallback-latency measurements derive from
`raw_input`, `evidence_refs`	The operator's original brief and artefact list — deliberately never mutated, so the self-evaluator can grade against the true original
`route_decision`	Router verdict: mode, intent, selected agents, confidence, rationale
`agent_outputs`	Each specialist's validated JSON report, keyed by agent name
`threat_score`, `threat_band`	The unit's authoritative fused severity and its categorical band
`escalation_path`	Which branch the threat gate selected — stored as state, not merely implied by which node ran
`evaluation`, `refinement_count`	Self-critique verdict and the bounded loop counter
`warnings`, `errors`	Structured advisories and faults; an entry with `fatal=True` diverts the graph to fallback
`fallback_triggered`, `final_report`	The mission product and whether it was degraded
`trace`	Ordered execution log — the artefact shown during the live demonstration

2.4 The router is DYNAMIC — explicit declaration

> **Declaration required by the assessment brief (A2.2):** the SENTINEL-4 router is **DYNAMIC**. It performs LLM-driven semantic intent classification over the raw operator brief and selects a specialist set from the inferred intent. It is *not* a keyword matcher. This declaration appears in the source itself at `src/router.py:3–13`.

It is dynamic *with a deterministic safety net*. If the semantic pass fails — endpoint unreachable, unparseable reply, contract breach, or a selection the router knows to be unsafe — control falls to a static keyword matcher (`static_route`, `router.py:88`). The system therefore keeps the discriminating power of semantic routing without inheriting its single point of failure. Critically, the mode actually used is written to the blackboard as `DYNAMIC_SEMANTIC` or `STATIC_FALLBACK` and printed in every run, so the operator is never misled about which mechanism produced the dispatch.

2.4.1 Hallucinated agent-name defence

A semantic router can invent a specialist. `sanitise()` (`router.py:119`) intersects the model's proposal against the frozen set `DISPATCHABLE_AGENTS` and unconditionally re-adds `cyber_coordinator`, because the escalation gate and the predictor both depend on it. A discarded name raises a structured warning rather than a `KeyError` deep inside dispatch.

2.5 Model-misrouting guard (A2.3)

The assessment brief specifically warns that inference calls must not be misrouted to embedding models such as `text-embedding-nomic-embed-text-v1.5`. SENTINEL-4 treats this as a configuration error to be caught before startup, not a runtime surprise. `ModelRegistry` (`config.py:60`) validates the chat slot in `__post_init__` against a table of embedding-name markers (`embed`, `nomic-embed`, `text-embedding`, `bge-`, `gte-`, `e5-`) and refuses to construct. `resolve("chat")` re-checks on every outbound call at `llm_client.py:460`.

Verification. Deliberately misconfiguring the chat slot produces:

```
$ MAS_PROVIDER=local MAS_LOCAL_CHAT_MODEL=text-embedding-nomic-embed-text-v1.5 python main.py --brief "test"

Configuration error: Refusing to start: 'text-embedding-nomic-embed-text-v1.5' is configured
as the chat model but its name identifies it as an embedding model.
Set MAS_*_CHAT_MODEL to an instruct/chat model id.
```

Regression-tested at `tests/test_routing.py:75`.

2.6 The four conditional routing paths

#	Gate	Condition	Source
1	`dispatch_gate`	Router intent determines *which* media specialists activate; if no media modality is present, fan directly to the coordinator	`graph.py:194`
2	`health_gate`	IF a fatal error exists, OR the 90 s mission deadline is breached, OR the fusion agent itself is `DEGRADED` → `fallback`; ELSE → `threat_fusion`	`graph.py:210`

#	Gate	Condition	Source
3	`escalation_gate`	**IF fused threat_score ≥ 80 THEN** `strategic_predictor` **ELSE** `standard_defense` — the brief's worked example, implemented literally	`graph.py:232`
4	`evaluation_gate`	REFINE with loop budget remaining and deadline intact → back to `threat_fusion`; ACCEPT → `synthesise`; fatal error → `fallback`	`graph.py:273`

Both branches of Path 3 are exercised by the shipped scenarios and asserted in `tests/test_routing.py:101`. Path 1 producing genuinely different agent sets for different briefs is demonstrated in Section 6.1.

3. Agent Role & Prompt Design Log

3.1 Role separation principle

The five agents are mutually exclusive *by construction*, not by convention. Every persona carries an explicit **STRICT BOUNDARIES** block that names what belongs to someone else. This was a deliberate design response to the most common multi-agent failure we anticipated: agents that quietly duplicate one another's reasoning, producing an assessment that looks corroborated but rests on one piece of evidence counted three times.

Agent	Role tag	Owns	Explicitly forbidden	Tools
Deepfake Audio Analysis Specialist 虚假语音识别专员	`AUDIO_FORENSICS`	Speech-audio authenticity; synthesis indicators; operational risk of the impersonation	Video, network telemetry, strategy	`scan_audio_artifacts` `parse_indicators`
Manipulated Video Stream Detector 流视频编辑检测专员	`VIDEO_FORENSICS`	Frame-level manipulation; distinguishing manipulation from misrepresentation	Audio authenticity, containment	`check_frame_consistency` `parse_indicators`
Cyber Offense/Defense Coordinator 网络攻防专员	`CYBER_OPS`	Evidence fusion; attack-vector *verification*; containment actions executable within the hour	Re-litigating media verdicts; forward-looking strategy	`query_threat_intel` `parse_indicators`
Tactical & Strategic Predictor 敌方战术战略预判及应对策略设计专员	`STRATEGY`	Adversary next-moves; counter-measures that can be pre-positioned *before* the move	Restating containment; planning on an unverified vector	`query_threat_intel`
Standard Defense Posture	`DEFENSE`	Proportionate routine hardening for sub-threshold incidents	Escalation of any kind	`parse_indicators`

3.2 Format constraints — enforced on every agent (A3.11)

All five specialists inherit one shared contract, `FORMAT_CONTRACT_TEMPLATE` (`agents/base.py:32`), appended automatically by `system_prompt()`. The router (`router.py:59`) and the self-evaluator (`evaluation.py:44`) carry equivalent contracts of their own. Centralising the constraint means no agent can drift from the standard:

```
OUTPUT FORMAT — NON-NEGOTIABLE
You must reply with a single raw JSON object and nothing else.
Do not wrap it in markdown fences. Do not add commentary before or after it.
```

```
The object must contain exactly these keys: {keys}.
Numeric fields must be plain integers between 0 and 100 with no units or symbols.
List fields must be JSON arrays of short strings.
If evidence is insufficient for a field, still emit the key and state the
limitation inside the string value — never omit a key and never return null.
```

The contract is enforced, not merely requested. `complete_json()` (`llm_client.py:508`) verifies every key in `required_keys` is present and raises `LLMContractError` otherwise — a typed failure the agent catches and converts into a `DEGRADED` report rather than letting `None` poison the blackboard.

Agent	Contracted output keys
Router	`intent`, `selected_agents`, `confidence_score`, `rationale`, `next_step`
Audio Analyst	`analysis_result`, `authenticity_verdict`, `threat_score`, `confidence_score`, `indicators`, `next_step`
Video Detector	`analysis_result`, `manipulation_verdict`, `threat_score`, `confidence_score`, `indicators`, `next_step`
Cyber Coordinator	`analysis_result`, `attack_vector`, `vector_verified`, `threat_score`, `confidence_score`, `containment_actions`, `next_step`
Strategic Predictor	`analysis_result`, `predicted_next_moves`, `counter_strategy`, `threat_score`, `confidence_score`, `next_step`
Standard Defense	`analysis_result`, `counter_strategy`, `threat_score`, `confidence_score`, `next_step`
Self-Evaluator	`analysis_result`, `coverage_score`, `unmet_requirements`, `verdict`, `confidence_score`, `next_step`

3.3 Few-shot prompting (A3.10)

The Audio Analyst and the Video Detector each carry **two complete Input/Output example pairs**; the Cyber Coordinator and the Strategic Predictor carry one each. The two-example agents pair one *positive* case with one *negative* case, and that pairing is the point.

> **Why two examples, and why one of them must be negative.** A single high-confidence example teaches the model what a detection looks like but never what a non-detection looks like. The observed consequence, recorded at `agents/audio_analyst.py:8–11`, was an agent that returned `SYNTHETIC` for every artefact presented to it and never once exercised the `INCONCLUSIVE` branch — a detector with a 100% true-positive rate and a 100% false-positive rate, which is to say no detector at all. Adding a second example built on weak evidence (a short, noisy sample with one indicator) gave the model a template for restraint.

Audio Analyst — Few-shot Example 2 (the restraint case)

```
Input: "Eight-second lobby recording, heavy background noise. Tool evidence:
synthesis_likelihood 41, indicators [prosody_flattening]."
Output:
{"analysis_result": "A single weak indicator on a short, noisy sample is
insufficient to separate synthesis from codec artefacts; no action-bearing
content is present.", "authenticity_verdict": "INCONCLUSIVE", "threat_score": 22,
"confidence_score": 45, "indicators": ["prosody_flattening",
"insufficient_sample_duration"], "next_step": "request_longer_sample"}
```

Video Detector — Few-shot Example 2 (manipulation vs. misrepresentation)

The video agent's negative example teaches a distinction the audio channel does not have: pixels can be entirely authentic while the *framing* is a lie. The example shows footage with no manipulation artefacts but an embedded timestamp twelve months stale, and models the correct verdict — `AUTHENTIC` pixels, elevated threat score, `flag_misrepresentation_to_cyber_coordinator`. Without it the agent collapsed both cases into a binary deepfake / not-deepfake judgement and under-reported the second.

3.4 Prompt refinement log

The table records each prompt-level intervention, the failure that motivated it, and where the result lives in the source. [Group: replace the "Observed effect" column with your own measured before/after outputs where you captured them — the entries below describe the behaviour the change was designed to correct, as recorded in the source annotations.]

Iteration	Problem observed	Prompt change applied	Observed effect
P1 Audio Analyst	Returned `SYNTHETIC` for every artefact; `INCONCLUSIVE` branch never fired	Added a second few-shot pair built on deliberately weak evidence; added an explicit VERDICT VOCABULARY block defining `SYNTHETIC` (≥2 independent indicators), `INCONCLUSIVE` (exactly one, or degraded evidence) and `AUTHENTIC`	Verdict now tracks indicator count rather than defaulting to the alarming answer

Iteration	Problem observed	Prompt change applied	Observed effect
P2 Audio Analyst	Threat score tracked how *synthetic* the audio sounded, not how dangerous it was	Added: "Base `threat_score` on operational danger (who is impersonated, what action is being requested), NOT merely on how synthetic the audio sounds"	A convincing clone asking for nothing now scores below a crude clone requesting a wire transfer
P3 Video Detector	Authentic footage presented in a false context was scored as harmless	Added the MANIPULATION vs. MISREPRESENTATION distinction plus a worked negative example with a stale embedded timestamp	Agent now reports authentic pixels with an elevated threat score and flags the framing
P4 Video Detector	On voice-only incidents the agent speculated about video that did not exist	Added an explicit null-verdict instruction: return `NO_VIDEO_EVIDENCE` with `threat_score` 0 when the brief has no visual component	Idle-modality hallucination eliminated; the fusion step receives an honest absence
P5 Cyber Coordinator	Declared vectors "verified" on a single specialist's say-so	Added FUSION RULES: `vector_verified` may be true only when ≥2 *independent* evidence sources agree (e.g. a specialist verdict plus a threat-intel corpus hit)	Verification became a corroboration test rather than a restatement
P6 Cyber Coordinator	Threat scores drifted around the 80 escalation boundary, making the branch unstable	Added: "A score of 80 or above escalates the mission to predictive planning. Do not cross 80 casually, and do not stay under it to avoid the work."	Named the threshold and its consequence inside the prompt so the number is a decision, not a guess
P7 Cyber Coordinator	Containment lists drifted into long-term programmes ("establish a governance board")	Constrained to "three or fewer imperative actions an operator can execute within the hour. No advisory language."	Output became a duty-officer checklist rather than a consultancy deliverable
P8 Strategic Predictor	Restated the Coordinator's containment actions as if they were strategy	Added boundary: "You do NOT restate containment actions. The Coordinator owns the present tense; you own the next 72 hours." Every predicted move must be paired with a *pre-positionable* counter-measure	Eliminated duplication between the two agents' outputs
P9 Strategic Predictor	Planned confidently against vectors the Coordinator had not verified	Added a HARD DEPENDENCY block: if `vector_verified` is false, state that planning is blocked, cap `confidence_score` at 20, set `next_step` to `await_vector_verification`	The dependency required by A2.7 is now enforced at the prompt layer as well as the graph layer
P10 All agents	Markdown-fenced JSON and conversational preambles broke parsing	Centralised `FORMAT_CONTRACT_TEMPLATE`; added the explicit no-fence, no-commentary, no-null, never-omit-a-key clauses	Parse failures fell to the residual cases the repair ladder in `extract_json` handles

Iteration	Problem observed	Prompt change applied	Observed effect
P11 Router	Activated media specialists on incidental mentions (“they contacted the helpdesk”)	Added ROUTING DOCTRINE: activate a media specialist only when the modality is actually present; idle specialists “cost latency and dilute the fused assessment with null findings”	Dispatch sets became genuinely input-dependent — see Section 6.1
P12 Self-Evaluator	Approved almost everything; the refinement loop never fired	Reframed as adversarial: “You are an adversarial reviewer, not a cheerleader. You did not write the draft and you gain nothing from approving it.” Added four named grading criteria and banned vague criticism	Evaluator now returns REFINE with specific named gaps — observed live in Section 6.1

3.5 Persona excerpt — Cyber Offense/Defense Coordinator

Reproduced from `src/agents/cyber_coordinator.py:43–83`; the shared format contract from `agents/base.py:32` is appended to this body at runtime.

```
ROLE-TAG: CYBER_OPS
You are the Cyber Offense/Defense Coordinator of countermeasure unit SENTINEL-4,
operating against the rogue autonomous system codenamed "The Entity".

MISSION SCOPE
Fuse the media-forensics findings of your fellow specialists with network and
host telemetry into ONE verified attack vector, and specify the containment
actions that stop it now.

STRICT BOUNDARIES
- You do not re-litigate media authenticity. Accept the audio and video verdicts
  as given, and weight them by the confidence_score each specialist reported.
- You handle CONTAINMENT (stopping what is happening). You do NOT produce
  predictive strategy — that belongs to the Tactical & Strategic Predictor.
- Never invent a tool. Your only tools are: {tool catalogue injected at runtime}

FUSION RULES
- Set "vector_verified" true only when at least two independent evidence sources
  agree (for example: a specialist verdict plus a threat-intelligence corpus hit).
- Your "threat_score" is the unit's authoritative figure. Anchor it on the
  highest specialist score, then adjust for blast radius and reversibility.
- A score of 80 or above escalates the mission to predictive planning. Do not
  cross 80 casually, and do not stay under it to avoid the work.
- "containment_actions" must be three or fewer imperative actions an operator can
  execute within the hour. No advisory language, no long-term programmes.

FEW-SHOT EXAMPLE (verified multi-vector intrusion)
Input: "Audio specialist: SYNTHETIC, confidence 88. Video specialist:
NO_VIDEO_EVIDENCE. Intel: 185.220.101.44 is a known Entity C2 node, severity 88.
Helpdesk reset MFA for two privileged accounts after the call."
Output:
{"analysis_result": "Synthetic-voice pretexting obtained an MFA reset, and the
resulting session beacons to a corpus-confirmed Entity C2 node. Media forensics
and network intelligence agree, so the vector is verified.", "attack_vector":
"voice_pretexting -> MFA_reset -> valid_account_access -> C2_beacon",
"vector_verified": true, "threat_score": 87, "confidence_score": 86,
```

```
"containment_actions": ["Revoke sessions and re-enrol MFA for both accounts",
"Blackhole 185.220.101.44 at the perimeter", "Freeze helpdesk credential resets
pending out-of-band verification"], "next_step": "escalate"}
```

3.6 Tool advertisement inside the prompt

Each persona interpolates `registry.catalogue_for(self.name)`, so an agent is told exactly which tools it owns and nothing more. This is paired with the instruction "Never invent a tool." The prompt-level constraint is backed by a runtime one: `ToolRegistry.invoke` (`tools/registry.py:93`) raises `HallucinatedToolError` for any unregistered name, which the agent absorbs as an evidence-quality warning. Prompt discipline and runtime enforcement are deliberately redundant, because prompt discipline alone is probabilistic.

4. Workflow & Communication Strategy

4.1 Pattern declaration

> **Declaration required by the assessment brief (A2.6):** SENTINEL-4 implements a **hierarchical supervisor over a shared-state blackboard**. A dynamic supervisor fans work out to media specialists that run concurrently; every report lands on one `MissionState`; a fusion agent reads the whole board and verifies an attack vector; a conditional gate then decides whether predictive planning is warranted. Declared in source at `src/graph.py:3-16`.

This is a hybrid of two classical patterns rather than a textbook instance of either. The supervisor relationship (Manager–Worker) governs *who runs*; the blackboard (Erman et al., 1980; Hayes-Roth, 1985) governs *how findings are shared*. Separating those two questions is what allows the dispatch set to vary per incident while the data-sharing contract stays fixed.

4.2 Dependency handling (A2.7)

The brief requires that "the Strategic Predictor cannot formulate a plan until the Cyber Coordinator verifies the attack vector." SENTINEL-4 enforces this at three layers, deliberately redundantly:

Layer	Mechanism	Source
Topological	Both media specialists have unconditional edges into `cyber_coordinator`. LangGraph super-step semantics make the coordinator wait for every branch that actually started, and the predictor is only reachable through the coordinator and the fusion node.	`graph.py:331-332`
Data	`gather_evidence()` reads `vector_verified` and `attack_vector` off the shared blackboard before the predictor's own LLM call is made — the dependency is a genuine data read, not merely an ordering convention.	`agents/strategic_predictor.py:205-225`
Semantic	The HARD DEPENDENCY prompt block forces the predictor to declare planning blocked and cap its confidence at 20 if the vector arrives unverified.	`agents/strategic_predictor.py:172-176`

The health gate adds a fourth safeguard: if the coordinator itself is `DEGRADED`, the graph diverts to fallback rather than letting downstream agents build on a fusion point that failed (`graph.py:224-229`). Continuing would produce a confident-looking empty assessment, which is strictly worse than an honest partial one.

4.3 Justification against the alternatives (A2.6)

Alternative considered	Why it was rejected for this problem
Sequential pipeline (CrewAI-style linear crew)	The audio and video specialists are genuinely independent — neither consumes the other's output. Serialising them makes wall-clock time the *sum* of two analyses instead of the

Alternative considered	Why it was rejected for this problem
	maximum, for no analytical benefit. It also encodes the dependency on the Coordinator only as position in a list, which is invisible to a reader and unenforced at runtime.
Pure point-to-point message passing	The Predictor's dependency is a *data* dependency, not a message-ordering one. On a blackboard, the predictor reads `vector_verified` off shared state and refuses to plan without it — one guarantee, implemented once. With point-to-point messages, that guarantee has to be re-implemented in every sender, and the fallback handler has no single place to look for salvageable findings.
Free-form group chat (AutoGen-style conversable agents)	An open conversation has no natural termination condition, which makes both the loop guard and the 5-second fallback budget unenforceable. Bounded execution is a hard requirement of the risk-mitigation clause, and an explicit graph provides it. The trade-off is less emergent behaviour, which this domain does not want: a countermeasure unit should be auditable and repeatable, not creative.
Single agent with many tools	Fails the assessment's architectural requirement, but also fails on the merits: one context window holding audio forensics, video forensics, network telemetry and strategic doctrine produces cross-contamination, and there is no point at which an independent verification step can be inserted.

4.4 Communication mechanics

Agents never call one another. Each returns a *blackboard patch* — a partial `MissionState` — which LangGraph merges using the declared reducers. The base class guarantees the shape of that patch and, critically, guarantees it is always produced: `SpecialistAgent.run()` (`agents/base.py:158`) catches the transport's entire error hierarchy and returns a `DEGRADED` report rather than raising into the graph.

> **Invariant.** An agent never raises into the graph. This single rule is what makes the partial-response fallback possible: the orchestrator can always assume there is *something* on the board to salvage, and the fusion step can decide what is trustworthy rather than crashing on what is missing.

4.5 Worked execution trace

Actual output of `python main.py --scenario scenarios/scenario_multi_vector.json`:

```
 1. ingest             Mission SENTINEL-5E42D1CD accepted (810 chars, 2 artefact(s))
 2. router             Routing mode DYNAMIC_SEMANTIC -> ['audio_analyst', 'video_detector',
                                                        'cyber_coordinator']
 3. audio_analyst      Deepfake Audio Analysis Specialist filed report (OK)
 4. video_detector     Manipulated Video Stream Detector filed report (OK)
 5. cyber_coordinator  Cyber Offense/Defense Coordinator filed report (OK)
 6. threat_fusion      Fused threat score 96.11 (CRITICAL)
 7. escalation_gate    score 96.11 vs threshold 80.0: ESCALATED -> predictive planning
 8. strategic_predictor Tactical & Strategic Predictor filed report (OK)
 9. self_evaluation    Verdict REFINE (coverage 70.0, loop 0)
10. threat_fusion      Fused threat score 96.3 (CRITICAL)
11. escalation_gate    score 96.3 vs threshold 80.0: ESCALATED -> predictive planning
12. strategic_predictor Tactical & Strategic Predictor filed report (OK)
13. self_evaluation    Verdict ACCEPT (coverage 91.0, loop 1)
14. synthesise         Full assessment released                      [78 ms total]
```

Steps 3–5 demonstrate the dependency: two specialists file concurrently, and the coordinator runs only after both have landed. Steps 9–13 demonstrate the self-evaluation loop closing — the evaluator rejected its own

unit's first draft at coverage 70, the workflow re-entered fusion, and the second draft was accepted at coverage 91. Step 13 also shows the loop terminating by *verdict* rather than by exhausting its budget, which is the healthy case.

5. Robustness & Fallback Analysis

5.1 Failure taxonomy and response

Failure mode	Detection	System response
Endpoint unreachable / timeout	`LLMUnavailableError` normalised from any transport exception (`llm_client.py:472`)	Bounded exponential-backoff retry; on exhaustion the calling agent files `DEGRADED`; if it was the fusion agent, the graph diverts to fallback
Malformed / fenced JSON	`json.JSONDecodeError` caught in the three-strategy repair ladder (`llm_client.py:97`)	Parse as-is → unwrap markdown fence → extract outermost brace span; one retry with a blunt corrective suffix; then `LLMParseError`
Schema drift (missing key)	Required-key check after parse (`llm_client.py:572`)	`LLMContractError`; the agent degrades rather than propagating `None` into the blackboard
Hallucinated tool	`HallucinatedToolError` at registry resolution (`tools/registry.py:113`)	Refused and logged; agent proceeds on partial evidence with a `tool_degradation` warning
Hallucinated tool *arguments*	`TypeError` separated from generic tool faults (`tools/registry.py:118`)	Reported distinctly as `bad_arguments` — a prompt-design signal, not a code bug
Hallucinated agent name	Set intersection against `DISPATCHABLE_AGENTS` (`router.py:134`)	Silently corrected; a structured warning records what was discarded
Non-numeric score	`ValueError` / `TypeError` / `AttributeError` in `_coerce_score` (`evaluation.py:72`)	Coerced to 0.0; handles `"87"`, `"87/100"`, `"87%"` and `"high"`
Missing peer report	`KeyError` on the state dict (`agents/cyber_coordinator.py:106`)	Recorded as `NOT_DISPATCHED` — an absence, not a fault
Infinite reasoning loop	`GraphRecursionError` at `recursion_limit=24` (`graph.py:388`)	Caught by the outermost net; converted to a partial response built from the last streamed state
Wall-clock exhaustion	90 s mission deadline checked in two gates (`graph.py:221`, `graph.py:289`)	Diverts to fallback or forces finalisation — time and iterations are treated as separate exhaustion modes
Embedding model in chat slot	`ModelRoutingError` at construction (`config.py:83`)	Refuses to start with a specific remediation message
Malformed env var	`ValueError` in `_env_float` (`config.py:178`)	Falls back to documented default — an operator typo should not prevent startup

5.2 Quantified budgets (A3.4, A3.5)

Requirement	Configured	Enforcement
Timeout ≤ 30 s	25 s default	Clamped by `_env_float(..., maximum=30.0)` at `config.py:240`; applied both on the client object (`llm_client.py:433`) and per request (`llm_client.py:469`), so no call site can issue an unbounded request

Requirement	Configured	Enforcement
max_retries ≤ 3	2 default (3 total attempts)	Clamped at `config.py:241`; the retry loop is implemented in-house (`OpenAI(max_retries=0)`) specifically so every attempt is logged
Back-off strategy	0.4 s → 0.8 s → 1.6 s, capped at 4 s	`llm_client.py:553`. A flat delay against a rate-limited endpoint burns the retry budget without letting the limiter recover; the cap keeps the total inside the 30 s ceiling

Both ceilings are regression-tested at `tests/test_routing.py:82` — the test sets deliberately out-of-range environment values and asserts they are clamped.

5.3 Loop prevention — three independent guards

A single loop guard is a single point of failure. SENTINEL-4 uses three that fail independently:

1. **Refinement budget** — `max_refinement_loops = 2`. When the evaluator requests REFINE with no budget left, the verdict is rewritten to `ACCEPT_BUDGET_EXHAUSTED` and an explicit warning is attached (`evaluation.py:231–240`). The product is released with a caveat rather than cycling.
2. **Graph recursion limit** — 24 super-steps (`graph.py:49`). Catches loops the refinement counter cannot see, such as a cycle between nodes that never touches the evaluator.
3. **Mission deadline** — 90 s wall clock (`graph.py:53`). Catches the case where a slow endpoint burns the entire budget inside a *legal* refinement loop: the counter says "keep going" while the operator waits. This guard was added after review specifically because time and iteration count are separate exhaustion modes (`graph.py:290–295`).

5.4 Fallback design (A2.10)

Two properties make the < 5 s guarantee real rather than aspirational:

- **No LLM calls.** `src/fallback.py` is pure Python over the blackboard. It cannot fail for the same reason that triggered it — a fallback path that itself needs the dead endpoint is not a fallback. This is why it completes in single-digit milliseconds.
- **No invention.** Every line of a fallback product is copied from a report that actually completed. `salvage_findings()` (`fallback.py:328`) excludes `DEGRADED` reports outright. Where nothing completed, the product says so and recommends no action at all.

The degraded product carries five things: a headline risk warning, the trigger, one de-duplicated operator-guidance entry per distinct failure class, a completion percentage measured against agents actually dispatched, and an explicit re-run instruction.

> **Design intervention.** The initial implementation stopped at echoing whatever survived. An explicit re-run instruction was appended (`fallback.py:167–169`) because a partial product with no stated next step reads, to a tired operator at 3 a.m., like a complete one. The degraded product must be visibly degraded at the point of use, not merely in its metadata.

A second intervention makes the salvage meaningful at all. The orchestrator executes the graph with `graph.stream()` rather than `graph.invoke()` (`graph.py:376–380`), because `invoke()` discards everything the graph produced when it raises — a loop breach at step 23 threw away 22 steps of good analysis and left the fallback with nothing. Streaming and retaining the newest emitted state makes the partial response genuinely partial rather than empty.

5.5 Fusion under degradation

`fuse_threat_score()` (`evaluation.py:81`) *excludes* degraded reports and renormalises the remaining weights over the survivors, rather than counting a failed specialist as zero. Counting a failure as zero would drag the fused score down and could silently de-escalate a genuine emergency — **absence of evidence is not evidence of safety**. Asserted at `tests/test_routing.py:108`.

Agent	Fusion weight	Rationale
`cyber_coordinator`	0.50	Dominates because it alone sees the full evidence picture
`audio_analyst`	0.20	Single-modality evidence
`video_detector`	0.20	Single-modality evidence
`strategic_predictor`	0.10	Forward-looking projection, not observed evidence

5.6 Measured fallback evidence (A3.7)

Produced by `python demo_fallback.py`. Each case injects a real failure through the client's injection channels (`llm_client.py:418–420`) — no code is modified to stage them.

Case	Injected failure	System behaviour	Product	Latency
1	Endpoint outage at the Cyber Coordinator (fusion point)	3 attempts consumed, then diverted to fallback; audio + video findings salvaged	`PARTIAL_RESPONSE` 66.7% complete	**1 234 ms**
2	Malformed non-JSON output from the Audio Analyst	That agent degrades and is excluded from fusion; mission continues with 4 agents	`FULL_ASSESSMENT` fused 92.71	**78 ms**
3	Hallucinated tool `run_nmap_scan`	Refused at the registry; `tool_degradation` warning raised; agent completes normally on real evidence	`FULL_ASSESSMENT`	**63 ms**
4	Infinite reasoning loop (forced cycle)	`GraphRecursionError` at limit 24 caught by the outermost net; last coherent state salvaged	`PARTIAL_RESPONSE`	**156 ms**

> **Result:** all four failure classes produce a useful operator-facing product. The slowest is 1 234 ms — **4× inside the assessment's 5-second ceiling** — and that figure is dominated by the deliberate retry back-off, not by the fallback path itself, which `tests/test_fallback.py:67` asserts completes in under 100 ms. **No unhandled exception occurred in any case.**

Sample degraded product (Case 1)

```
!! RISK WARNING — DEGRADED OUTPUT — do not treat as a complete assessment.
 Trigger    : LLMUnavailableError in cyber_coordinator
 Completion : 66.7%
 Failure    : LLMUnavailableError in cyber_coordinator
              -> Inference endpoint unreachable. Treat the findings below as
                 UNCONFIRMED and re-run once the endpoint is restored.

 SALVAGED FINDINGS
  - audio_analyst: Synthetic speech signature detected: prosody flattening and
    absent glottal micro-jitter consistent with a neural vocoder.
  - video_detector: Frame-level inconsistency detected: illumination vector
    diverges from scene geometry across the facial region.

 FALLBACK STRATEGY
  - [audio_analyst] forward_to_cyber_coordinator
  - [video_detector] forward_to_cyber_coordinator
  - Re-run the full workflow before committing further resources
    (current confidence is partial at fused score 0.0).
```

5.7 Exception-handler inventory (A3.6)

The codebase contains **18 exception handlers across 14 distinct exception types**, against an assessment minimum of three. Handlers target named failure points rather than wrapping code in blanket `except Exception`:

#	Exception	Location	Failure point targeted
1	`json.JSONDecodeError`	`llm_client.py:97`	LLM output that is not valid JSON — treated as control flow in the repair ladder
2	`Exception` → `LLMUnavailableError`	`llm_client.py:472`	Transport layer: timeout, refused connection, rate limit, 5xx — normalised into one typed error
3	`HallucinatedToolError`	`agents/base.py:126`	Agent requests a tool that does not exist
4	`KeyError`	`agents/cyber_coordinator.py:106`	Peer report absent from the shared state dictionary
5	`Exception` (outermost net)	`graph.py:388`	`GraphRecursionError` and anything unforeseen → partial response
6	`TypeError`	`tools/registry.py:118`	LLM invents *arguments* for a real tool
7	`(ValueError, TypeError, AttributeError)`	`evaluation.py:72`	Non-numeric threat score, e.g. `"high"`
8	`LLMError` hierarchy	`agents/base.py:202`	Any transport failure inside an agent → `DEGRADED` report
9	`LLMError`	`router.py:181`	Semantic routing failure → static keyword fallback
10	`LLMError`	`evaluation.py:208`	Evaluator failure → fails *open* with an explicit unreviewed caveat
11	`ValueError`	`config.py:178`	Malformed environment variable
12	`ImportError`	`config.py:27`, `llm_client.py:436`	Optional dependency absent
13	`json.JSONDecodeError`, `FileNotFoundError`	`main.py:41–46`	Malformed or missing scenario file — clean exit, no traceback in front of an audience
14	`ToolExecutionError`	`agents/base.py:131`	Registered tool raised while executing

> **Deliberate narrowness.** The handler in `agents/base.py:202` was narrowed from a generated `except Exception` to the transport's own error hierarchy, so that genuine programming bugs (`TypeError`, `AttributeError`) still surface loudly in development instead of being silently laundered into a degraded report that looks like a routine API outage. Rationale recorded at `agents/base.py:203–207`.

6. Evaluation Results

6.1 Scenario results — dynamic routing and conditional branching

Three scenarios were designed to exercise different paths through the graph. The differing dispatch sets are the primary evidence that routing is input-dependent rather than fixed, and the differing escalation paths are the evidence that Conditional Path 3 works in both directions.

Scenario	Agents dispatched	Fused score	Band	Escalation branch
`scenario_multi_vector` audio + video + network	audio_analyst, video_detector, cyber_coordinator	96.30	CRITICAL	`strategic_predictor` (IF ≥ 80)
`scenario_audio_only` voice-only, no visual	audio_analyst, cyber_coordinator — video specialist correctly left idle	96.71	CRITICAL	`strategic_predictor`
`scenario_low_threat` benign report	cyber_coordinator only	5.00	MINIMAL	`standard_defense` (ELSE branch)

> **Both sides of the escalation condition are demonstrated**, and the router produces three different dispatch sets from three different briefs — including correctly leaving the video specialist idle on a voice-only incident, which was the behaviour prompt iteration P4 and P11 targeted.

6.2 Automated test suite

```
$ python -m pytest tests/ -q
.................................                                        [100%]
35 passed in 4.79s
```

Module	Tests	Coverage focus
`tests/test_routing.py`	13	Dynamic dispatch, idle-specialist suppression, static degradation, hallucinated agent names, mandatory coordinator, embedding-model refusal, budget clamping, both escalation branches, degraded-report exclusion, non-numeric score handling
`tests/test_tools.py`	11	Registry minimum, hallucinated tool refusal, bad-argument reporting, IOC extraction per class, IP-octet false-positive regression, determinism, empty-input rejection, intel hit/miss, path-traversal blocking, missing-file handling
`tests/test_fallback.py`	11	Endpoint outage → partial response, fallback latency budget (< 100 ms and < 5 s), malformed JSON survival, contract-breach classification, recursion ceiling, refinement budget, empty-salvage safe advice, warning de-duplication, all three scenarios reaching their documented branch

6.3 Requirement self-audit

`audit_requirements.py` measures the running system against the assessment's quantifiable requirements. It is included so that a grader can reproduce every claim in this report with one command.

```
$ python audit_requirements.py

 CHECK                          REQUIRED      RESULT  MEASURED
 ------------------------------------------------------------------------
 Specialist agents              >= 3          PASS    5 registered
 Custom tools                   >= 3          PASS    5 registered
 Conditional routing paths      >= 2          PASS    4 (dispatch, health, escalation, evaluation)
   - escalation branch works    both sides    PASS    strategic_predictor / standard_defense
   - dispatch varies by intent  yes           PASS    agent set differs between briefs
 Router is dynamic              LLM-driven    PASS    DYNAMIC_SEMANTIC
 LLM timeout                    <= 30s        PASS    25.0s
 LLM max retries                <= 3          PASS    2
 try/except blocks              >= 3          PASS    21 handlers, 14 exception types
 Docstring coverage             100%          PASS    100.0%
 [HUMAN-REVIEW] comments        >= 5          PASS    13 found
 Fallback emits partial output  yes           PASS    PARTIAL_RESPONSE
 Fallback latency               < 5s          PASS    1234 ms
 Infinite-loop guard            caught        PASS    GraphRecursionError at limit 24
 ------------------------------------------------------------------------
 14/14 checks passed.
```

Independent verification: an AST walk over the *entire* submitted tree — including `tests/`, `demo_fallback.py` and `audit_requirements.py` — confirms docstrings on **173 of 173** modules, classes, methods and functions (100%). Restricting the same walk to `src/` alone gives **18 `except` handlers across 14 distinct exception types**. The audit script reports different absolute numbers only because it deliberately scopes itself to `src/` plus the root scripts and excludes the test suite and itself; both scopes are 100% documented and both exceed the required handler threshold.

6.4 Annotation coverage (A3.8, A3.9)

Metric	Required	Achieved	Evidence
Docstring coverage (PEP 257)	100%	100% (173/173)	Every class, method and tool function documents purpose, Args, Returns and Raises
Human-curated `[HUMAN-REVIEW]` comments	≥ 5	13	Enumerated in Section 7.3; reproduce with `grep -rn "HUMAN-REVIEW" src/`
Module-level design documentation	—	All 17 modules	Pattern declaration, router-type declaration and fallback philosophy are documented in the source, not only in this report

7. AI Usage & Code Generation Log

> This section is submitted in accordance with the module's AI-Assisted Development Policy, which permits and expects the use of Generative AI to write core application code, and assesses architectural rationality, robustness, prompt-engineering quality and annotation sufficiency rather than manual coding syntax. It is a full and honest declaration.

7.1 Tools used

Tool	Used for	How its output was evaluated
[NAME THE ASSISTANT USED FOR IMPLEMENTATION] Phase 1 — implementation	Generation of the application source in `src/`: LangGraph assembly, the abstract agent base class and five personas, the five custom tools, the resilient LLM transport layer, the fallback module and the test suite.	Architectural decisions — collaboration pattern, reducer strategy, role separation and the three-layer dependency enforcement — were specified by the group *before* generation, and the output was checked against that specification. Every generated module was executed against the test suite before acceptance. Generated logic was read line by line; the 13 points at which it was rejected or altered are documented individually in §7.3.
Claude Opus (Anthropic) via the Claude Code CLI Phase 2 — verification & documentation	Independent audit of the finished codebase against the assessment brief; production of this report, the architecture diagram (`docs/architecture.svg`) and the presentation deck.	Nothing in this report is asserted from reading the code alone. Every quantitative claim was re-derived by *executing* the submitted system: the 35-test suite was run, all three scenarios were executed end to end, `demo_fallback.py` was run and timed, docstring coverage was recounted by an independent AST walk (173/173), exception handlers were enumerated, and the embedding-model guard was triggered live to confirm it refuses to start. The figures in §5.6 and §6 are measured outputs, not estimates. The table-of-contents page numbers were extracted from the rendered PDF and verified section by section.
LM Studio local instruct model (`qwen2.5-7b-instruct`) [delete this row if you never ran against a local model]	Runtime inference backend for testing the agent prompts against a real, non-deterministic model rather than the offline mock engine.	Used to surface real-model failure modes that a deterministic mock cannot reproduce — notably the non-numeric `threat_score` defect (a 7B model answering `"high"` instead of an integer) recorded at `evaluation.py:73-76`, which produced the score-coercion guard.

AI assistance fell into two distinct phases, and this report separates them deliberately rather than reporting a single undifferentiated figure. **Phase 1** produced the application code and is the phase the module's policy is chiefly concerned with; the group's review of that output is evidenced by the thirteen annotated interventions in §7.3. **Phase 2** did not write application code at all — it audited the finished system by running it, and produced the documentation you are reading. Where the two phases disagreed, the executed result was taken as authoritative: §6.3 records a case where the project's own audit script and an independent count reported different totals, and the report states both scopes rather than quietly adopting the more flattering one.

7.2 Division of labour between human and AI

Decided by the group (human)	Generated by AI, then reviewed
Choice of theme and adversary model	Persona prose and few-shot example text
Collaboration pattern (blackboard vs. pipeline vs. group chat)	LangGraph wiring that implements it
Which agent owns which responsibility, and the boundary statements	Prompt phrasing of those boundaries
Where the escalation threshold sits and what it gates	Gate implementation and branch recording
The rule that agents must never raise into the graph	Base-class implementation of that invariant
The rule that the fallback path makes no network calls	`fallback.py` implementation
Fusion weights and the exclude-don't-zero rule for degraded reports	Weighted-fusion arithmetic

7.3 Human review interventions

Every location where the group rejected or altered generated logic is annotated in the source with a `[HUMAN-REVIEW]` comment explaining the reasoning. All thirteen are reproduced below; run `grep -rn "HUMAN-REVIEW" src/` to verify.

#	Location	What the AI produced	Why it was changed
1	`state.py:33`	Default last-write-wins on `agent_outputs`	Silently deleted the Audio analyst's report whenever Video finished in the same super-step. Replaced with an explicit merge reducer — the single most important correctness fix in the state layer.
2	`graph.py:376`	`graph.invoke()`	`invoke()` discards everything the graph produced when it raises, so a loop breach at step 23 threw away 22 steps of good analysis. Switched to `stream()` retaining the newest state, making the partial response genuinely partial.
3	`agents/base.py:203`	Bare `except Exception`	Narrowed to the transport's own error hierarchy so genuine programming bugs still surface loudly instead of being laundered into a degraded report resembling a routine outage.
4	`graph.py:290`	Evaluation gate checked only the loop counter	A slow endpoint can burn the whole wall-clock budget inside a *legal* refinement loop. Time and

#	Location	What the AI produced	Why it was changed
			iterations are separate exhaustion modes and both must be able to stop the loop.
5	`router.py:129`	Trusted `selected_agents` verbatim	A hallucinated name such as `malware_reverse_engineer` produced a `KeyError` deep inside dispatch. Now intersected against `DISPATCHABLE_AGENTS`, turning a hard crash into a corrected decision.
6	`llm_client.py:548`	Flat 1-second retry sleep	A flat delay against a rate-limited endpoint burns the retry budget without letting the limiter recover. Replaced with capped exponential back-off that stays inside the 30 s ceiling.
7	`evaluation.py:73`	Direct `float()` on the model's score	A local 7B model returned `"threat_score": "high"`; the `ValueError` propagated out of the graph and killed a run that had already produced four good specialist reports.
8	`fallback.py:167`	Fallback echoed only what survived	A partial product with no stated next step reads, to a tired operator at 3 a.m., like a complete one. An explicit re-run instruction is now appended.
9	`tools/evidence_reader.py:43`	Checked for the literal string `".."` in the filename	Defeated by symlinks and absolute paths. Replaced with a resolved-path containment check — the only form of this guard that actually holds.
10	`tools/indicator_parser.py:63`	Domain regex matched IPv4 octet pairs	`"10.20.30.40"` yielded a spurious domain `"30.40"`. Domains that are substrings of already-extracted IPs are now subtracted, removing all false positives in the test corpus.
11	`tools/registry.py:119`	Single generic tool-failure handler	`TypeError` separated deliberately: an LLM inventing *arguments* for a real tool is a prompt-design problem, and must be reported distinctly from a bug inside the tool body.
12	`config.py:28`	`python-dotenv` as a hard import	Crashed the whole system when run without the package installed.

#	Location	What the AI produced	Why it was changed
			Downgraded to a soft dependency so real OS environment variables still work and the MAS starts everywhere.
13	`config.py:179`	Malformed env var raised at startup	`MAS_REQUEST_TIMEOUT="thirty"` is an operator typo, not a system fault. Now falls back to the documented default rather than refusing to run.

7.4 What AI output was rejected outright

- **Blanket exception handling.** Generated code defaulted to `except Exception` in several places. Only two survive — both deliberate outermost safety nets — and each is annotated as such.
- **Optimistic fallback behaviour.** Early generated fallback logic produced a plausible-looking full report from missing data. This was replaced with the strict no-invention rule in `salvage_findings()`.
- **Single-example prompting.** Generated personas carried one worked example each. The group added negative examples to the two media agents after observing the always-positive verdict failure described in Section 3.3.

7.5 Academic-integrity position

The group understands the material in this project. The architecture was specified before generation, every generated line was reviewed, thirteen substantive corrections are documented in the source with their reasoning, and the group can explain and modify any part of the system on request. No architectural design was copied from another group.

8. Repository Link & User Guide

8.1 Repository

Repository URL	https://github.com/DHARIZMAN/sentinel4-mas
Default branch	`main`
Secrets policy	No API keys are committed. `.env`, `*.key` and `secrets/` are listed in `.gitignore`; only `.env.example` with placeholder values is tracked.

8.2 Prerequisites

- Python 3.10 or newer (developed and tested on 3.12)
- No GPU, no API key and no network connection required — the default provider is a deterministic offline engine

8.3 Installation

```
git clone https://github.com/DHARIZMAN/sentinel4-mas.git
cd entity_mas

python -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate

pip install -r requirements.txt
cp .env.example .env              # Windows: copy .env.example .env
```

8.4 Running the system

Command	What it demonstrates
`python main.py --scenario scenarios/scenario_multi_vector.json`	All three specialists dispatched, fused score ≥ 80, **escalated** branch, self-evaluation refinement loop
`python main.py --scenario scenarios/scenario_audio_only.json`	Dynamic router activates **only** the audio specialist — video correctly left idle
`python main.py --scenario scenarios/scenario_low_threat.json`	Score < 80 → **Standard Defense** branch (the ELSE path)
`python main.py --brief "Cloned voice call requesting an MFA reset" --evidence call.wav`	Ad-hoc free-text input
`python main.py --scenario scenarios/scenario_multi_vector.json --json`	Machine-readable mission product

8.5 Demonstrating the fallback (required for the live demo)

```
python demo_fallback.py            # all four failure classes, each timed
python demo_fallback.py --case 1   # endpoint outage only
python demo_fallback.py --case 4   # infinite-loop / recursion ceiling only
```

8.6 Tests and self-audit

```
python -m pytest tests/ -v          # 35 tests
python audit_requirements.py        # measures the code against the assessment rubric
```

8.7 Docker

```
docker build -t sentinel4-mas .
docker run --rm sentinel4-mas
docker run --rm --env-file .env sentinel4-mas --scenario scenarios/scenario_low_threat.json
```

8.8 Connecting a real language model

Set `MAS_PROVIDER` in `.env`:

Value	Configuration
`mock` (default)	Deterministic offline engine. No key, no network. Used for reproducible grading and for the automated tests.
`local`	LM Studio / Ollama / vLLM. Set `MAS_LOCAL_BASE_URL` (default `http://localhost:1234/v1`) and `MAS_LOCAL_CHAT_MODEL` to a **chat/instruct** model id.
`remote`	Any hosted OpenAI-compatible endpoint. Set `MAS_REMOTE_BASE_URL`, `MAS_REMOTE_API_KEY`, `MAS_REMOTE_CHAT_MODEL`.

> **Model naming is strictly managed.** `MAS_*_CHAT_MODEL` must name an instruct/chat model. Supplying an embedding model id — for example `text-embedding-nomic-embed-text-v1.5` — causes the system to refuse to start with a specific remediation message, by design (Section 2.5).

8.9 Troubleshooting

Symptom	Cause and remedy
`ModuleNotFoundError: langgraph`	Virtual environment not activated, or dependencies not installed. Re-run `pip install -r requirements.txt`.
`Configuration error: Refusing to start…`	An embedding model id is configured in the chat slot. Set `MAS_*_CHAT_MODEL` to an instruct model.
Routing mode shows `STATIC_FALLBACK`	The semantic pass failed — usually the local endpoint is not running. The system is working as designed; start the endpoint to restore dynamic routing.
Process exits with code 2	Not an error: exit code 2 signals that the run degraded to a partial response. Exit code 0 signals a full assessment.

9. Limitations & Future Work

Limitation	Discussion and proposed work
Forensic tools are simulations	`scan_audio_artifacts` and `check_frame_consistency` derive deterministic, plausible findings from artefact references rather than performing real signal processing. This was a deliberate scope decision: it keeps the project runnable on any grading machine with no media dependencies while exercising exactly the same agent/tool contract a production detector would use. `parse_indicators` and `read_evidence_file` do perform real work. Future work: replace the two simulators with real detectors behind the identical `ToolSpec` interface — no agent or graph code would change.
Default provider is an offline mock	The deterministic engine makes grading reproducible, but in mock mode the router's semantic pass is served by a keyword-driven stand-in. The dynamic code path is genuine and exercised whenever a real model is attached. Future work: ship a recorded transcript of a `local`-provider run alongside the mock output so both are auditable without a running endpoint.
Fusion weights are fixed constants	`FUSION_WEIGHTS` encodes an analyst judgement (the coordinator sees the most, so it weighs most). It is not learned and not calibrated against ground truth. Future work: calibrate against a labelled incident corpus and expose the weights as configuration.
Self-evaluation shares the model that produced the draft	The adversarial framing mitigates but does not eliminate self-agreement bias. Future work: route the evaluator to a different model, or add a second evaluator and require consensus.
Single-mission scope	Each run is independent; there is no cross-incident memory, so a campaign spanning several briefs is not correlated. Future work: persist `MissionState` to a store keyed by campaign and let the coordinator query prior missions.
No human-in-the-loop checkpoint	The system runs to completion autonomously. In a real SOC, containment actions affecting production would require operator confirmation. Future work: LangGraph interrupt-before on the containment node.

10. Plagiarism / AI Declaration Statement

We, the undersigned members of this project group, declare that:

1. This project and report are our own work, produced for **ISB 46703 Principles of Artificial Intelligence**, and have not been submitted for any other assessment.
2. The architectural design of this Multi-Agent System — the collaboration pattern, agent role separation, routing strategy, escalation logic and fallback design — is our own and was **not copied from any other group**.
3. Generative AI tools were used to write core application code, as expressly permitted and expected by the module's AI-Assisted Development Policy. The tools used, what each was used for, and how their output was evaluated and modified are declared in full in **Section 7** of this report.
4. Every location at which we rejected or altered AI-generated logic is annotated in the submitted source code with a `[HUMAN-REVIEW]` comment recording our reasoning. There are thirteen such annotations, reproduced in Section 7.3.
5. All external sources, frameworks and literature consulted are acknowledged in the References section in APA 7th edition format.
6. We understand the system we are submitting and can explain, justify and modify any part of it on request.
7. We understand that plagiarism and undeclared AI use are serious academic offences, and that the penalties described in the institution's academic-integrity regulations apply.

No.	Full Name	Student ID	Signature	Date
1 (Leader)	HARRIS ILHAM SHAH BIN MOHD AZRAF	52213125949		
2	DHARIZMAN BIN GHAZALI @ HASSAN	52213125618		
3	AZAD ARIF BIN ABDUL RAZAK	52213125884		
4	MUHAMMAD NAZRUL AIMAN BIN MOHD NIZAM	52213124200		

Contribution statement

Member	Primary contribution
HARRIS ILHAM SHAH BIN MOHD AZRAF 52213125949 · Leader	Architecture & LangGraph assembly; dynamic router design; state management and concurrency reducers — [adjust to your actual split]
DHARIZMAN BIN GHAZALI @ HASSAN 52213125618	Agent personas & prompt engineering; few-shot design and the 12-iteration refinement log — [adjust to your actual split]
AZAD ARIF BIN ABDUL RAZAK 52213125884	Robustness layer, fallback / risk-warning module, loop guards and the test suite — [adjust to your actual split]

Member	Primary contribution
MUHAMMAD NAZRUL AIMAN BIN MOHD NIZAM 52213124200	Custom tools, threat-fusion and evaluation harness, scenarios and report production — [adjust to your actual split]

11. References

APA 7th edition.

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., … Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems, 33*, 1877–1901.

Erman, L. D., Hayes-Roth, F., Lesser, V. R., & Reddy, D. R. (1980). The Hearsay-II speech-understanding system: Integrating knowledge to resolve uncertainty. *ACM Computing Surveys, 12*(2), 213–253. https://doi.org/10.1145/356810.356816

Guo, T., Chen, X., Wang, Y., Chang, R., Pei, S., Chawla, N. V., Wiest, O., & Zhang, X. (2024). Large language model based multi-agents: A survey of progress and challenges. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI-24)* (pp. 8048–8057). https://doi.org/10.24963/ijcai.2024/890

Hayes-Roth, B. (1985). A blackboard architecture for control. *Artificial Intelligence, 26*(3), 251–321. https://doi.org/10.1016/0004-3702(85)90063-3

LangChain. (2025). *LangGraph documentation*. https://langchain-ai.github.io/langgraph/

MITRE. (2025). *MITRE ATT&CK® enterprise matrix*. The MITRE Corporation. https://attack.mitre.org/

Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. (2023). Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST '23)* (Article 2). https://doi.org/10.1145/3586183.3606763

Russell, S. J., & Norvig, P. (2021). *Artificial intelligence: A modern approach* (4th ed.). Pearson.

Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems, 36*, 8634–8652.

Verdoliva, L. (2020). Media forensics and DeepFakes: An overview. *IEEE Journal of Selected Topics in Signal Processing, 14*(5), 910–932. https://doi.org/10.1109/JSTSP.2020.3002101

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E. H., Le, Q. V., & Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems, 35*, 24824–24837.

Wooldridge, M. (2009). *An introduction to multiagent systems* (2nd ed.). John Wiley & Sons.

Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A. H., White, R. W., Burger, D., & Wang, C. (2023). *AutoGen: Enabling next-gen LLM applications via multi-agent conversation* (arXiv:2308.08155). arXiv. https://arxiv.org/abs/2308.08155

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. In *Proceedings of the Eleventh International Conference on Learning Representations (ICLR 2023)*. https://arxiv.org/abs/2210.03629

> **Before submission:** verify each reference against the source you actually consulted, and remove any you did not use. A reference list must reflect real reading.